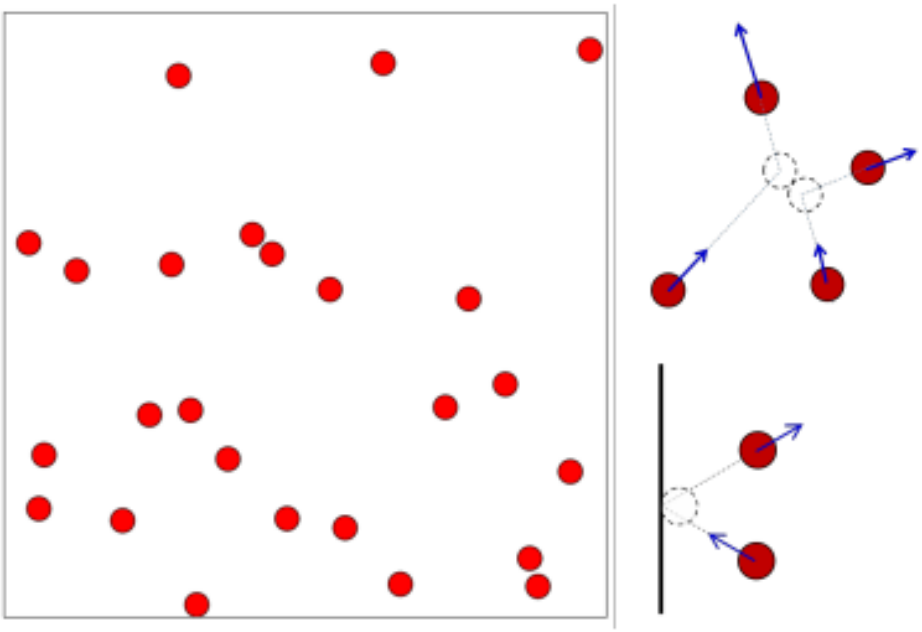
CR – Numérique

Modélisation d'un gaz de sphères dures



Auteurs :
Naji Amrani
Maxence Fillon
Rafael Jaubert–Martin

Promotion :
DMPM

Table des matières

1 Introduction **1**

2 Modélisation du gaz **1**
2.1 Hypothèses du modèle 1
2.2 Collision avec la paroi 1
2.3 Collision entre particules 2

3 Méthode numérique **3**
3.1 Détection des collisions 3
3.2 Calcul des données physiques 4

4 Distribution des vitesses **4**
4.1 Mise hors équilibre et objectif 4
4.2 Maxwell-Boltzmann en 2D 4
4.3 Résultats numérique et comparaison avec le modèle 5

5 Pression exercée sur les parois de la boite **5**
5.1 Définition mécanique de la pression 5
5.2 Méthode numérique 6
5.3 Résultats et comparaison avec le modèle 7
5.3.1 Méthode 1 : Répétitions identiques 7
5.3.2 Méthode 2 : Pression en fonction de la température 7

6 Mouvement brownien d'une particule plus grosse que les autres **7**
6.1 Principe physique 7
6.2 Mise en œuvre numérique 8
6.3 Critère de validité 8
6.4 Résultats et discussion 8

7 Conclusion **9**

8 Annexes **10**
8.1 Figures 10
8.1.1 Distribution de Maxwell-Boltzmann 10
8.1.2 Pression numérique 12
8.1.3 Mouvement brownien 14
8.2 Code 16
8.2.1 Structure principale du code 16
8.2.2 Animation 18
8.2.3 Distribution de Maxwell-Boltzmann 25
8.2.4 Pression Numérique 29
8.2.5 Mouvement Brownien 34

1 Introduction

Dans ce projet, nous modélisons le comportement d'un gaz parfait bidimensionnel confiné dans une boîte de surface S. Puisque nous nous restreignons à deux dimensions, nous utiliserons la notion de surface S plutôt que celle de volume V (ou 2-volume). Le gaz est modélisé par un ensemble de sphères dures de rayon r, de masse m et de vitesse $\boldsymbol{v}$, interagissant uniquement par des collisions élastiques entre elles et avec les parois de la boîte. L'objectif de ce projet est de vérifier numériquement plusieurs résultats fondamentaux de la physique statistique : la distribution de Maxwell-Boltzmann, la loi des gaz parfaits et le mouvement brownien, à l'aide de simulations numériques. Les résultats obtenus seront comparés aux prédictions théoriques afin de vérifier ces lois dans le cadre du modèle considéré. Dans un premier temps, nous expliquerons la physique des collisions ainsi que l'approche numérique utilisée. Nous présentons ensuite les résultats numériques obtenus, en analysant dans l'ordre suivant : la distribution de Maxwell-Boltzmann, la loi des gaz parfaits et le mouvement brownien d'une particule de taille plus grande que les autres.

2 Modélisation du gaz

2.1 Hypothèses du modèle

Nous faisons les hypothèses suivantes pour modéliser notre gaz de sphères dures :

- Les particules sont des sphères dures de rayon r et de masse m.
- Les particules interagissent uniquement par des collisions élastiques entre elles et avec les parois de la boîte, l'énergie du système est conservée à chaque collision.
- Nous nous restreignons à deux dimensions, c'est à dire que les particules évoluent dans un plan, nous utiliserons donc la notion de surface S plutôt que celle de volume V.
- Le gaz est confiné dans une boîte de surface S avec des parois parfaitement rigides et immobiles.
- Les collisions sont instantanées, c'est à dire que le temps de collision est négligeable par rapport au temps entre deux collisions, nous pouvons donc considérer que les particules sont en mouvement libre entre deux collisions.
- Le gaz est dilué, c'est à dire que la distance moyenne entre les particules est beaucoup plus grande que leur diamètre $2r$.
- Les particules sont indiscernables, c'est à dire que nous ne pouvons pas différencier une particule d'une autre, elles sont toutes identiques.

2.2 Collision avec la paroi

Dans le cas d'une collision avec la paroi, la vitesse de la particule est modifiée très simplement : la composante de la vitesse perpendiculaire à la paroi est inversée tandis que la composante parallèle reste inchangée. En prenant la base (Oxy), avec O placé dans le coin inférieur gauche de la boite et les axes Ox et Oy parallèles aux parois de vecteurs unitaires $\boldsymbol{e_x}$ et $\boldsymbol{e_y}$ respectivement, nous pouvons exprimer la nouvelle vitesse $\boldsymbol{v'}$ après collision en fonction de la vitesse initiale $\boldsymbol{v} = (v_x, v_y)$ et de la paroi touchée :

$$\boldsymbol{v'} = \begin{cases} (-v_x, v_y) & \text{si la particule touche une paroi verticale} \\ (v_x, -v_y) & \text{si la particule touche une paroi horizontale} \end{cases} \tag{2.1}$$

La modélisation d'une telle relation est simple, nous passons à présent aux collisions inter particules.

2.3 Collision entre particules

Dans le cas d'une collision entre deux particules, la règle reste la même. C'est à dire que d'une part, l'énergie cinétique du système est conservée, et pour une base bien choisie, seule la composante normale est modifiée. Supposons que nous avons deux particules de masses m_1 et m_2, de vitesses initiales $\boldsymbol{v_1}$ et $\boldsymbol{v_2}$, et de positions $\boldsymbol{r_1}$ et $\boldsymbol{r_2}$. Nous pouvons définir la base $O'x'y'$ où $\boldsymbol{u_{x'}}$ est le vecteur unitaire normal à la surface de contact des deux particules et $\boldsymbol{u_{y'}}$ est le vecteur unitaire tangent à cette surface.

Commençons par calculer de façon générale les nouvelles vitesses de nos deux particules après une collision élastique. (Nous resterons dans la base Oxy dans un premier temps)
D'après la conservation de l'énergie cinétique, nous avons :

$$\frac{1}{2}m_1v_1^2 + \frac{1}{2}m_2v_2^2 = \frac{1}{2}m_1v_1'^2 + \frac{1}{2}m_2v_2'^2 \tag{2.2}$$

$$\Leftrightarrow m_1v_1^2 + m_2v_2^2 = m_1v_1'^2 + m_2v_2'^2 \tag{2.3}$$

D'autre part, d'après la conservation de la quantité de mouvement, nous avons :

$$m_1\boldsymbol{v_1} + m_2\boldsymbol{v_2} = m_1\boldsymbol{v_1'} + m_2\boldsymbol{v_2'} \tag{2.4}$$

Supposons pour simplifier que les deux particules ont des vitesses colinéaires ($\boldsymbol{v_1}$ et $\boldsymbol{v_2}$ sont colinéaires). Alors l'équation de conservation de la quantité de mouvement peut être réécrite comme suit :

$$m_1v_1 + m_2v_2 = m_1v_1' + m_2v_2' \tag{2.5}$$

En combinant les deux équations, nous obtenons :

$$\begin{cases} v_1' = \frac{(m_1-m_2)v_1+2m_2v_2}{m_1+m_2} \\ v_2' = \frac{(m_2-m_1)v_2+2m_1v_1}{m_1+m_2} \end{cases} \tag{2.6}$$

Supposons également que les deux particules ont la même masse $m_1 = m_2 = m$, alors les équations précédentes se simplifient à :

$$\begin{cases} v_1' = v_2 \\ v_2' = v_1 \end{cases} \tag{2.7}$$

C'est à dire que les deux particules échangent leurs vitesses.

Généralisons à présent ces résultats au cas où les vitesses ne sont pas colinéaires. Dans ce cas, nous allons projeter les vitesses sur la base $O'x'y'$ que nous avons définie précédement. Nous allons noter $v_{1x'}$ et $v_{1y'}$ les composantes de la vitesse $\boldsymbol{v_1}$ de la première particule dans la base $O'x'y'$, et $v_{2x'}$ et $v_{2y'}$ les composantes de la vitesse $\boldsymbol{v_2}$ de la seconde particule dans la même base. Nous allons également noter $v'_{1x'}$ et $v'_{1y'}$ les composantes de la vitesse $\boldsymbol{v_1'}$ de la première particule après collision dans la base $O'x'y'$, et $v'_{2x'}$ et $v'_{2y'}$ les composantes de la vitesse $\boldsymbol{v_2'}$ de la seconde particule après collision dans la même base. D'après la conservation de l'énergie cinétique, nous avons :

$$\frac{1}{2}m_1(v_{1x'}^2 + v_{1y'}^2) + \frac{1}{2}m_2(v_{2x'}^2 + v_{2y'}^2) = \frac{1}{2}m_1(v_{1x'}'^2 + v_{1y'}'^2) + \frac{1}{2}m_2(v_{2x'}'^2 + v_{2y'}'^2) \tag{2.8}$$

$$\Leftrightarrow m_1(v_{1x'}^2 + v_{1y'}^2) + m_2(v_{2x'}^2 + v_{2y'}^2) = m_1(v_{1x'}'^2 + v_{1y'}'^2) + m_2(v_{2x'}'^2 + v_{2y'}'^2) \tag{2.9}$$

D'autre part, d'après la conservation de la quantité de mouvement, nous avons :

$$m_1\boldsymbol{v_1} + m_2\boldsymbol{v_2} = m_1\boldsymbol{v_1'} + m_2\boldsymbol{v_2'} \tag{2.10}$$

En projetant cette équation sur la base $O'x'y'$, nous obtenons :

$$\begin{cases} m_1 v_{1x'} + m_2 v_{2x'} = m_1 v'_{1x'} + m_2 v'_{2x'} \\ m_1 v_{1y'} + m_2 v_{2y'} = m_1 v'_{1y'} + m_2 v'_{2y'} \end{cases} \tag{2.11}$$

En combinant les deux équations, nous obtenons :

$$\begin{cases} v'_{1x'} = \frac{(m_1 - m_2) v_{1x'} + 2 m_2 v_{2x'}}{m_1 + m_2} \\ v'_{1y'} = v_{1y'} \\ v'_{2x'} = \frac{(m_2 - m_1) v_{2x'} + 2 m_1 v_{1x'}}{m_1 + m_2} \\ v'_{2y'} = v_{2y'} \end{cases} \tag{2.12}$$

C'est à dire que seule la composante normale de la vitesse est modifiée lors de la collision, tandis que la composante tangentielle reste inchangée.
Remarquons que dans le cas où les deux particules ont la même masse $m_1 = m_2 = m$, les équations précédentes se simplifient à :

$$\begin{cases} v'_{1x'} = v_{2x'} \\ v'_{1y'} = v_{1y'} \\ v'_{2x'} = v_{1x'} \\ v'_{2y'} = v_{2y'} \end{cases} \tag{2.13}$$

et les deux particules échangent simplement leurs composantes normales de vitesse, tandis que leurs composantes tangentielles restent inchangées.

3 Méthode numérique

3.1 Détection des collisions

La détection des collisions est l'étape cruciale dans notre simulation, car elle permet de déterminer quand et où les particules entrent en contact les unes avec les autres ou avec les parois de la boîte. De plus cette étape est centrale dans les différentes mesures que nous allons faire, notamment pour la mesure de la pression exercée sur les parois de la boîte. Pour détecter les collisions entre les particules, nous avons différentié deux approches : l'approche par pas de temps et l'approche par événements.

La première approche consiste à fixer (ou bien déterminer) un pas de temps Δt et à faire évoluer les positions et les vitesses des particules à chaque pas de temps en vérifiant à chaque fois s'il y a eu une collision entre deux particules ou entre une particule et la paroi. Cependant, cette approche est très coûteuse en termes de temps de calcul, et il faut vérifier à chaque pas de temps toutes les paires de particules pour détecter les collisions. C'est donc un problème de l'ordre de $O(N^2)$ où N est le nombre de particules. Par conséquent, pour un grand nombre de particules, cette approche devient rapidement impraticable. Elle se révèle néanmoins très utile pour l'animation du système, car elle permet de stocker les positions des particules à chaque pas de temps, ce qui est nécessaire pour créer une animation fluide.

La seconde approche, que nous avons choisie, consiste à calculer directement les instants de collision entre les particules et les parois, puis à faire évoluer le système de collision en collision. Pour ce faire, nous partons du système à l'instant $t = 0$ et nous calculons les trajectoires des particules (notons que ce sont des mouvements rectilignes uniformes entre les collisions) puis

nous enregistrons toutes les collisions potentielles et nous stockons les instants de collision dans une liste. Ainsi on sélectionne le temps le plus cours et on fait avancer le système jusqu'à ce temps de collision, puis on met à jour les vitesses des particules impliquées dans la collision et on réitère le processus. Cette approche est bien plus efficace que la première, car elle permet de sauter les intervalles de temps où il n'y a pas de collision, ce qui réduit considérablement le nombre de calculs nécessaires.

3.2 Calcul des données physiques

Selon l'objectif visé, nous avons besoin de calculer différentes données physiques à partir de notre simulation numérique. Pour la distribution de Maxwell-Boltzmann, nous avons besoin de calculer les vitesses des particules à différents instants de temps afin de construire l'histogramme de la distribution des vitesses et ainsi vérifier la loi. Pour la loi des gaz parfaits, nous avons besoin de calculer la pression exercée sur les parois de la boîte, ce qui nécessite de compter le nombre de collisions entre les particules et les parois ainsi que les impulsions échangées lors de ces collisions. Enfin, pour le mouvement brownien d'une particule plus grosse que les autres, nous avons besoin de suivre la trajectoire de cette "grosse" particule au cours du temps afin de calculer sa position et sa vitesse. Pour ce faire, nous avons mis en place des fonctions spécifiques dans notre code qui permettent de calculer ces différentes données physiques. Les codes utiliseront toujours des codes sources puis des fonctions spécifiques pour calculer les données physiques nécessaires à l'analyse des résultats.

4 Distribution des vitesses

4.1 Mise hors équilibre et objectif

Nous voulons étudier le retour à l'équilibre thermodynamique de notre système en partant d'une configuration initale hors équilibre. Pour ce faire, nous initialisons nos particules avec des vitesses arbitraire qui ne suivent aucune loi de distribution particulière en particulier celle de Maxwell-Boltzmann. Les vitesses étants choisies aléatoirement, l'état initial du système est hors équilibre et nous pouvons commencer notre expérience. Nous rappelons que d'après nos hypothèse du modèle, l'énergie du sytème est conservée à chaque collision que ce soit avec la paroi ou entre particules. Cependant, les collisions entre particules permettent une redistribution de l'énergie entre elles ce qui va permettre au système de redistribuer l'énergie dans le sytème. Nous attendons alors, d'un point de vue thermostatistique, que le système tende vers un état d'équilibre et qu'en même temps la distribution des vitesses évolue vers la distribution de Maxwell-Boltzmann. Pour observer cela nous mesurons la distribution des vitesses pour un intervalle régulier puis nous construisons un histogramme pour chaque instant mesuré afin de voir l'évolution temporelle de la ditribution des vitesses et ainsi comparer avec la loi théorique. Nous remarquerons l'existence d'un régime transitoire pendant lequel la distribution des vitesse varie fortement avant de se stabiliser vers notre distribution de Maxwell-Boltzmann.

4.2 Maxwell-Boltzmann en 2D

A l'équilibre thermodynamique, la théorie cinétique des gaz parfaits prédit que la distribution des composantes des vitesses des particules suit une loi gaussienne. En particulier dans notre cas bidimensionnel, la densité de probabilité pour les composantes des vitesses v_x et v_y est donnée par la formule suivante :

$$\begin{cases} f(v_x) = \sqrt{\frac{m}{2\pi k_B T}} \exp\left(-\frac{m v_x^2}{2 k_B T}\right) \\ f(v_y) = \sqrt{\frac{m}{2\pi k_B T}} \exp\left(-\frac{m v_y^2}{2 k_B T}\right) \end{cases} \tag{4.1}$$

et il s'en suit que la distribution de la norme de la vitesse $v = \sqrt{v_x^2 + v_y^2}$ est donnée par la formule suivante :

$$f(v) = \frac{mv}{k_B T} \exp\left(-\frac{mv^2}{2k_B T}\right) \tag{4.2}$$

De plus le théorème d'équipartition de l'énergie implique que la température T du système est liée à l'énergie cinétique moyenne des particules par la relation suivante :

$$\frac{1}{2} m\langle v^2 \rangle = k_B T \tag{4.3}$$

où $\langle v^2 \rangle$ est la moyenne de la norme de la vitesse au carré. On obtient alors la relation :

$$T = \frac{m\langle v^2 \rangle}{2k_B} \Leftrightarrow \langle v^2 \rangle = \frac{2k_B T}{m} \tag{4.4}$$

Le modèle théorique est établi et on le réutilisera pour l'analyse des données numériques.

4.3 Résultats numérique et comparaison avec le modèle

Pour simuler un gaz initialement hors équilibre, on attribue à chacune de nos 200 particules - disposée de manière aléatoire dans la boite selon une loi uniforme - une composante v_x et v_y tiré aléatoirement de manière uniforme sur un intervalle compris entre $-1000\ m \cdot s^{-1}$ et $1000\ m \cdot s^{-1}$. Ainsi, la distribution des composantes des vitesses n'est pas une densité normale de probabilité mais une densité uniforme, assurant que le système n'est pas à l'équilibre. On fait alors tourner notre simulation avec ces conditions initiales, et à pas de temps fixé on trace un histogramme de la distribution de la norme des vitesses, avec par dessus un graphe suivant la distribution de Boltzmann que devrait épouser la distribution des vitesses à l'instant du tracé si le système était à l'équilibre.Le code permettant d'effectuer la simulation est disponible en annexe.

Au lancement de la simulation, on observe que la distribution des données ne suit effectivement pas une distribution de Maxwell-Boltzmann (voir figure 1). Cependant, après 10 secondes de simulation, la distribution des vitesses suit globalement une distribution de Maxwell-Boltzmann, bien qu'imparfaite (figure 2). A partir d'un certain stade, la distribution ne se stabilise pas beaucoup plus autour de la distribution de Maxwell-Boltzmann et tend plutôt à "osciller autour" (figure 3). Cette difficulté de convergence peut néanmoins s'expliquer de manière assez convaincante. En premier lieu, notre système simulé n'est pas réaliste. Les particules ont une aire importante en comparaison de celle de la boite et elles sont très peu en comparaison du nombre d'Avogadro ($\sim 10^{23}$. On peut aussi mettre en cause le temps de la simulation, de seulement 20 secondes : plus le système s'approche de l'état idéal, plus les écarts résiduels deviennent faibles et nécessitent un temps de simulation croissant pour être réduits davantage. Ainsi, une simulation plus longue aurait permis d'observer une meilleure convergence.

5 Pression exercée sur les parois de la boite

5.1 Définition mécanique de la pression

La pression exercée par un gaz sur une paroi peut être définie mécaniquement comme le flux de quantité de mouvement transféré à cette paroi, par unité de temps et par unité de longueur.

Ainsi, si l'on observe le système pendant un intervalle de temps $t_{\text{simulation}}$, la pression moyenne est donnée par

$$P = \frac{1}{\mathcal{L}} \frac{\Delta p_{\text{total}}}{t_{\text{simulation}}},$$

où $\mathcal{L}$ désigne le périmètre total de la boîte, et Δp_{total} est la quantité de mouvement totale transmise aux parois pendant la durée de la simulation.

Lors d'une collision élastique entre une particule de masse m et une paroi, seule la composante de la vitesse perpendiculaire à la paroi est modifiée. Si $v_\perp$ désigne cette composante, alors après collision elle change de signe, ce qui entraîne une variation de quantité de mouvement :

$$\Delta p = m(-v_\perp) - m(v_\perp) = -2mv_\perp. \tag{5.1}$$

En valeur absolue, la quantité de mouvement transmise à la paroi lors d'un choc est donc

$$|\Delta p| = 2m|v_\perp|. \tag{5.2}$$

La quantité de mouvement totale transmise aux parois pendant la simulation est obtenue en sommant les contributions de toutes les collisions :

$$\Delta p_{\text{total}} = \sum 2m|v_\perp|. \tag{5.3}$$

La pression calculée numériquement correspond donc à une pression moyenne exercée sur l'ensemble du périmètre du domaine.

Par ailleurs, les vitesses initiales des particules sont tirées selon la distribution de Maxwell-Boltzmann. Le système est donc, dès l'initialisation, à l'équilibre thermodynamique statistique. En particulier, la distribution des directions de vitesse est isotrope, ce qui garantit que la pression moyenne exercée est identique sur toutes les parois.

5.2 Méthode numérique

Dans notre simulation, chaque collision entre une particule et une paroi est traitée individuellement. Nous enregistrons les données physiques utiles de la particule, c'est à dire sa masse et sa vitesse perpendiculaire à la paroi comme montré dans la section précédente. Nous calculons ensuite la contribution de cette collision à la pression en utilisant la formule précédemment définie. L'ensemble de toutes les contributions lors de la simulation est alors sommée pour obtenir la pression du système.

Cependant, la thermostatique est une discipline régie par les processus stochastiques, et la pression n'est pas une exception. Ainsi, les résultats que l'on obtiendra entre 2 simulations seront globalement différents, et il est peu probable que ces résultats soient tous très proches de la valeur théorique attendue donnée par la loi des gaz parfaits

$$P = \frac{Nk_BT}{S - N\pi r^2},$$

où N est le nombre de particules dans la boîte, k_B est la constante de Boltzmann, T est la température du système et $S - N\pi r^2$ représente l'aire (à la place du volume) de notre boîte soustraite de l'aire occupée par les particules, celui-ci n'étant dans notre modélisation non négligeable, en raison du rayon r important de nos particules. En clair, il est impossible de se prononcer sur le bon fonctionnement de notre modèle en faisant tourner quelques simulations tout au plus. On

préférera donc effectuer un grande nombre d'itérations de notre simulation, et observer si nos résultats convergent globalement vers la valeur théorique attendue.

Pour se faire, on procède de 2 manières différentes. Premièrement, on effectue un grand nombre de simulations avec les mêmes paramètres initiaux. En particulier, la température reste identique d'une simulation à l'autre, les particules étant disposées de manière aléatoire et la distribution de leur vitesses suivant une distribution de Maxwell-Boltzmann. On trace alors un histogramme des résultats obtenues au cours d'une centaine d'itérations. L'autre solution que l'on a trouvé est d'effectuer toujours un grand nombre de simulation (300), mais cette fois-ci en modifiant la température à chaque simulation. Nous pouvons - après une longue attente - tracer un graphe de la pression en fonction de la température, et voir si les valeurs numériques oscillent autour de la pression théorique. Les codes permettant d'effectuer ces 2 "simulations" (on entend par là la répétition contraintes des simulations simples) sont disponibles en annexe.

5.3 Résultats et comparaison avec le modèle

5.3.1 Méthode 1 : Répétitions identiques

La première "simulation" nous renvoie un histogramme des pressions, disponible en annexe. On observe que la distribution obtenue est globalement centrée sur la valeur théorique attendue, ce qui est un bon indicateur que notre simulation renvoie en moyenne une pression physiquement cohérente. L'étalement observé traduit simplement les fluctuations statistiques liées au fait que la pression est évaluée à partir d'un nombre fini de collisions durant un temps limité, faisant de la pression une valeur aléatoire dont l'espérance doit respecter les lois de la thermodynamique. Par ailleurs, cette "simulation" est une bonne manière d'observer l'aspect stochastique associé à la thermodynamique, et nous donne raison quant à la manière de tester notre modèle, via la répétition excessive - bien qu'elle pourrait l'être plus encore - de simulations.

5.3.2 Méthode 2 : Pression en fonction de la température

Après exécution du code associé, on obtient le graphe visible en annexe (figure 5). On observe alors que la pression numérique obtenue augmente de façon globalement linéaire avec la température, en accord avec la loi des gaz parfait, ce qui montre que la simulation reproduit correctement le comportement macroscopique attendu. Cependant, les valeurs ne suivent pas exactement la droite analytique, mais oscillent plutôt autour d'elle, et ce de manière plus intense au fur et à mesure que la température augmente. Cette dispersion est due au fait que notre manière de calculer la pression - une moyenne sur une longue période avec un petit nombre de particule (100) - est sujette à du bruit, qui croît avec la température. Néanmoins, l'ordre de grandeur est correct et aucune dérive systématique n'est à observer, nous confortant dans l'utilisation de ce modèle, et il est évident que l'écart observé pourrait être réduit en se rapprochant d'un système plus réaliste, i.e. en augmentant le nombre de particule et en diminuant leur rayon.

6 Mouvement brownien d'une particule plus grosse que les autres

6.1 Principe physique

Le mouvement brownien est un déplacement aléatoire d'une particule plus grosse et massive plongée dans un gaz ou un liquide constitué de particules plus petites. Ce mouvement résulte de l'agitation thermique des petites particules qui vont entrer en collision avec la grosse particule et induire un mouvement aléatoire. Mathématiquement, le mouvement brownien idéal peut être modéliser par un processus de Wiener $(W_t)_{t\geq 0}$ où $(W_t)_{t\geq 0}$ sera la position de la particule au cours du temps. Ce processus vérifie les propriétés suivantes :

— $W_0 = 0$
— les trajectoires sont presque sûrement continues
— les accroissements sont indépendants
— pour $0 \leq s < t$, on a :

$$W_t - W_s \sim \mathcal{N}(0, 2D(t-s)) \tag{6.1}$$

où D est le coefficient de diffusion (dans le cas du mouvement brownien).

La relation $\langle \Delta r^2(t) \rangle = 2Dt$, qui résulte de la dernière condition du processus de Wiener, est une équation caractéristique du mouvement brownien et explicite le caractère diffusif de ce mouvement.

6.2 Mise en œuvre numérique

Afin de pouvoir simuler un mouvement brownien nous introduisons une particule plus "grosse" (plus grande et plus massive). Nous faisons évoluer cette particule dans notre fluide en la munissant d'une vitesse initiale nulle et des mêmes aspects de collision qu'avec les particules du fluide.

Nous réaliserons plusieurs simulations totalement indépendantes afin d'obtenir des résultats statistiques réels. Les paramètres initiaux ont été ajusté afin d'avoir des résultats cohérent sans alourdir grandement le temps de calcul qui est déjà important. En particulier, afin d'augmenter l'efficacité de notre programme, nous enregistrerons uniquement les positions lors des collisions. Nous aurons donc un pas de temps variable. Ensuite, à partir de ces positions mesurées, il sera possible, pour un pas de temps donné, de calculer précisément les positions au cours du temps, puis les incréments de position, puisque entre chaque collision la particule suit un mouvement rectiligne uniforme.

6.3 Critère de validité

Pour vérifier que notre trajectoire issue de la simulation soit compatible avec le résultat théorique, nous testerons les propriétés du processus de Wiener citées dans la partie précédente. Nous analyserons la distribution des accroissements et les comparerons avec une distribution gaussienne qui est le modèle théorique.
Nous analyserons ensuite la dépendance de la variance des accroissements en fonction de l'intervalle de temps afin de vérifier la croissance linéaire de cette dépendance. Enfin, pour effectuer une étude complète, il conviendrait de calculer la covariance entre les accroissements successifs afin de s'assurer de l'absence de corrélation.
En effet, pour une distribution gaussienne, l'absence de corrélation implique l'indépendance des incréments. Cependant, par manque de temps, nous n'avons pas pu réaliser cette partie de l'analyse.

6.4 Résultats et discussion

Nous avons étudié la trajectoire de la grande particule dans un système composé de 500 particules. Comme le montre la figure en annexe (figure 6), la trajectoire obtenue présente, d'un point de vue qualitatif, les caractéristiques d'un mouvement brownien. Il apparaît donc pertinent, dans un premier temps, de vérifier si cette dynamique satisfait les conditions définissant un processus de Wiener.

Nous avons réalisé dix simulations indépendantes d'une durée de 30 secondes chacune, comportant 500 particules. Afin d'analyser les propriétés statistiques des déplacements, les incréments ont été calculés pour différents pas de temps, multiples de 0,001 s.

Pour un pas de temps fixé à 0,002 s, la distribution des incréments (figure 7) apparaît approximativement gaussienne et centrée en zéro. On observe néanmoins un pic plus marqué au centre de la distribution, ainsi que des queues légèrement plus épaisses que celles prédites par une loi normale idéale. Ces écarts peuvent être attribués à des effets numériques ou à des effets de bord liés au caractère fini de l'échantillon.

Enfin, en répétant l'analyse pour plusieurs valeurs du pas de temps et en calculant à chaque fois la variance des incréments correspondants, nous obtenons une relation linéaire entre la variance et le pas de temps (figure 8). Ce résultat est conforme à la propriété fondamentale du mouvement brownien selon laquelle :

$$\mathrm{Var}(\Delta X) \propto \Delta t$$

Les résultats obtenus sont compatibles avec le modèle théorique du mouvement brownien. En effet, nous observons une croissance linéaire de la variance — ou, de manière équivalente dans le cas du mouvement brownien, du déplacement quadratique moyen — en fonction du temps. Ce comportement est caractéristique de la nature diffusive du mouvement brownien.

Nous pouvons ainsi conclure que notre modèle numérique permet une étude qualitative pertinente du mouvement brownien.

7 Conclusion

Au cours de notre projet, nous avons étudié numériquement l'interaction d'un gaz de sphères dures confinées dans une boîte. Notre modèle repose sur des collisions élastiques entre les particules et avec les parois, ce qui nous a permis d'analyser l'évolution du système à partir d'un état initial hors équilibre.

Nous avons observé que le système évolue spontanément vers un régime stationnaire proche de l'équilibre. À partir de ce modèle microscopique, nous avons ainsi mis en évidence l'émergence de comportements macroscopiques caractéristiques de la physique statistique.

La distribution des vitesses converge vers une distribution de Maxwell-Boltzmann, en accord avec le modèle théorique. De plus, l'étude de la pression exercée sur les parois, déterminée à partir de la quantité de mouvement transmise par les particules, conduit à des résultats numériques cohérents avec le modèle des gaz parfaits. Les écarts observés peuvent s'expliquer par certaines hypothèses simplificatrices, notamment le fait que notre gaz simulé n'était pas assez proche des conditions d'un gaz parfait.

Enfin, nous avons établi un lien avec notre projet expérimental en étudiant le mouvement brownien. L'introduction d'une particule plus massive et plus grande que les autres nous a permis d'analyser ses déplacements et de mettre en évidence le caractère diffusif du mouvement, en accord avec le modèle de Wiener.

Ces simulations illustrent ainsi comment des interactions microscopiques simples peuvent conduire à des lois macroscopiques bien connues. Une extension naturelle de ce travail serait d'étudier la répartition de l'énergie du système ou encore de ne plus fixer la boîte, afin de la laisser se déplacer sous l'effet des collisions avec les particules.

8 Annexes

Note : La section qui suit est une annexe. Elle n'est pas à compter dans le nombre de page maximal du document autorisé. La présence de l'annexe dans le document à rendre est purement pratique, permettant une navigation plus pratique entre le compte rendu et les différents documents qui y sont liés.

8.1 Figures

Dans cette section vous trouverez l'ensemble des figures (histogrammes, graphes, ...) dont il est fait mention au cours de document. Les mentions des figures dans le texte sont cliquables et vous amèneront directement à la figure correspondante en annexe. De même, chaque figure possède un lien permettant de vous renvoyer à la section correspondante.

8.1.1 Distribution de Maxwell-Boltzmann

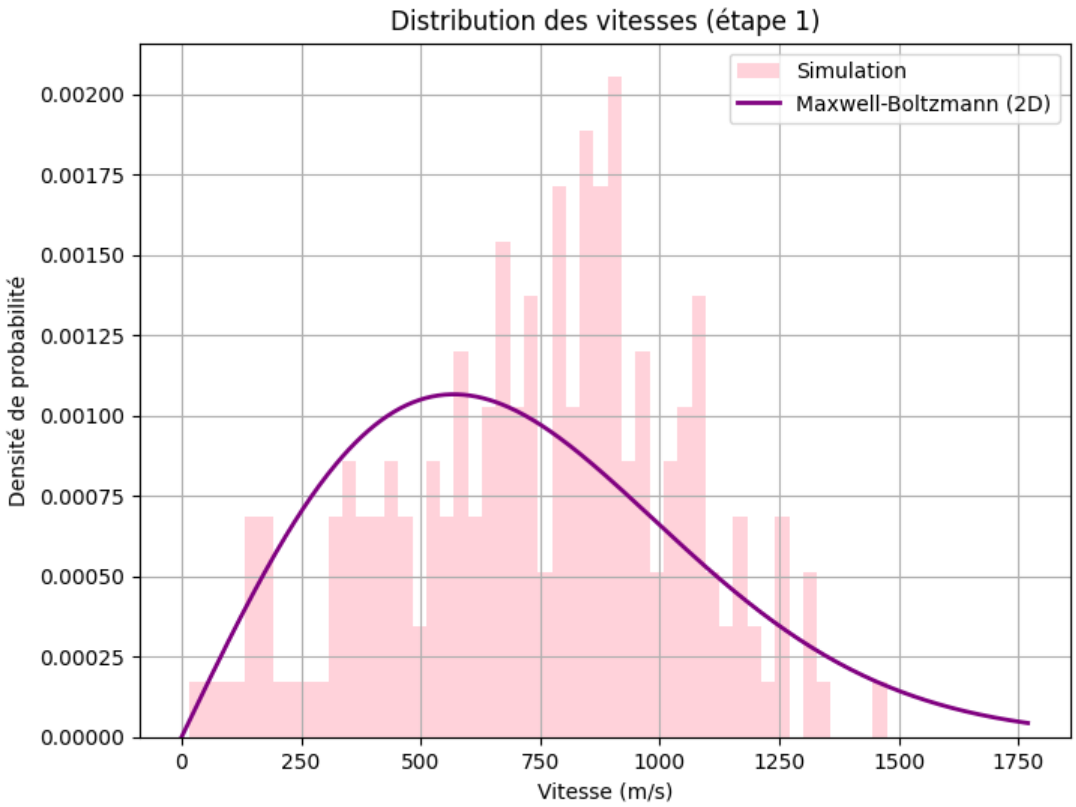


FIGURE 1 – Distribution initiale des vitesses d'un système hors équilibre thermodynamique

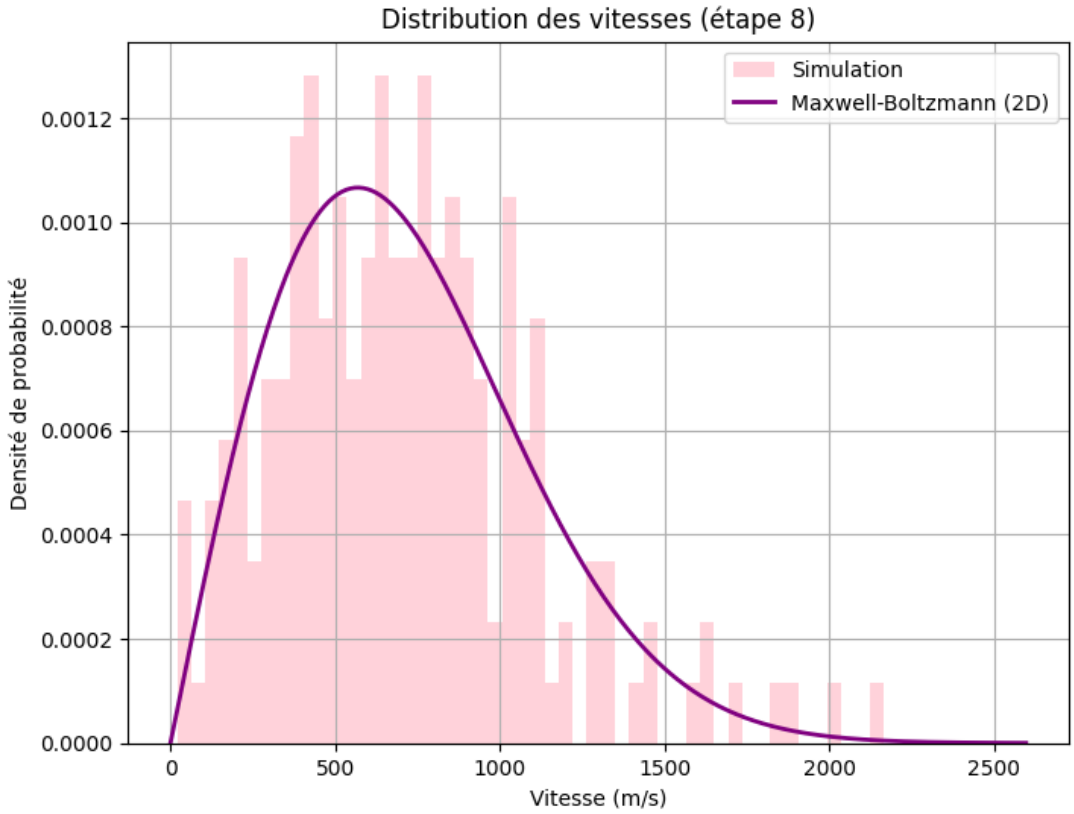


FIGURE 2 – Distribution des vitesses d'un système initialement hors équilibre thermodynamique après 10 secondes de simulation

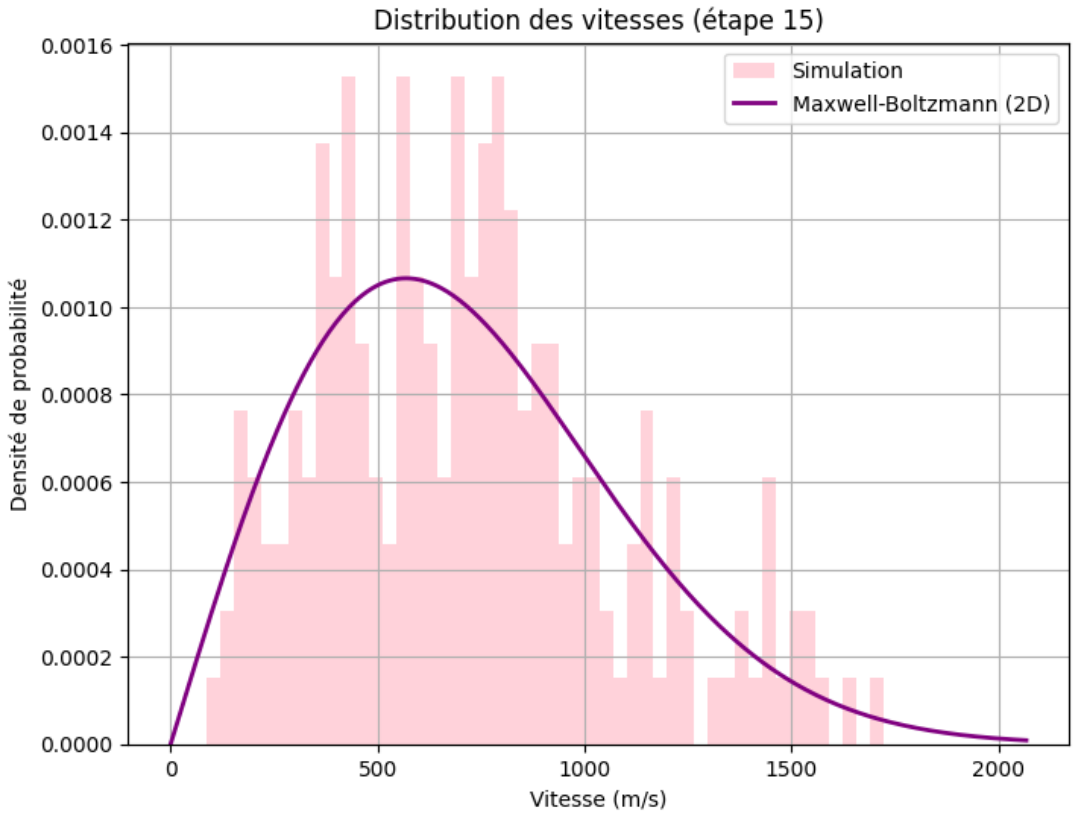


FIGURE 3 – Distribution des vitesses d'un système initialement hors équilibre thermodynamique à la fin de la simulation

8.1.2 Pression numérique

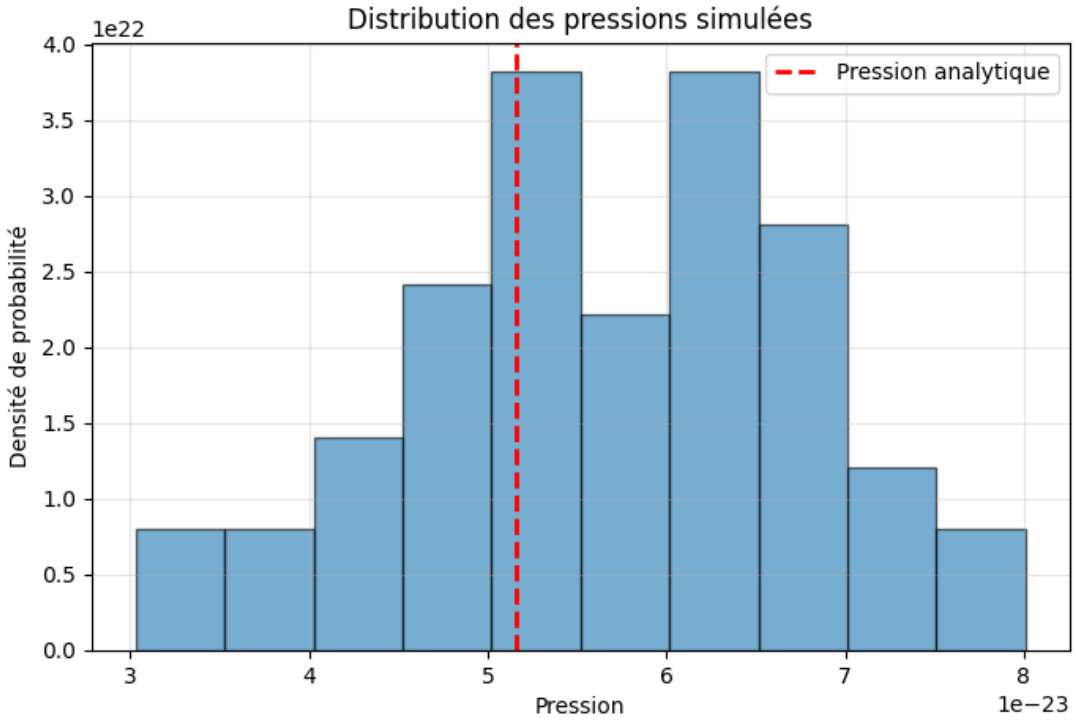


FIGURE 4 – Distribution des pressions numériques calculées 100 fois dans les mêmes conditions pour un gaz simulé

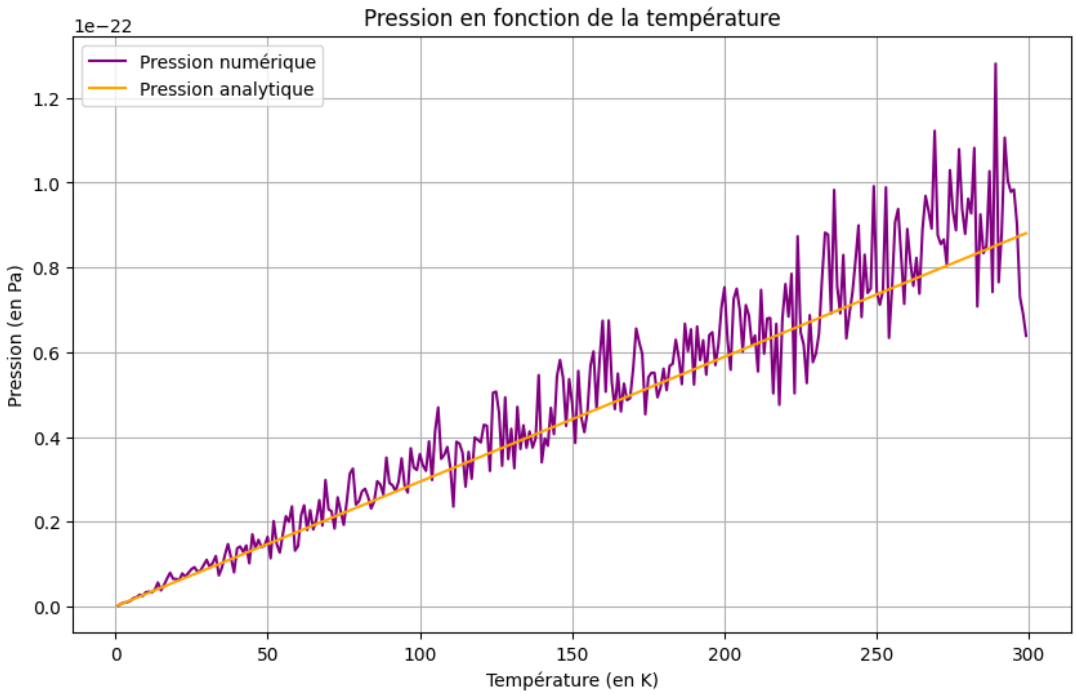


FIGURE 5 – Pression en fonction de la température pour un gaz simulé

8.1.3 Mouvement brownien

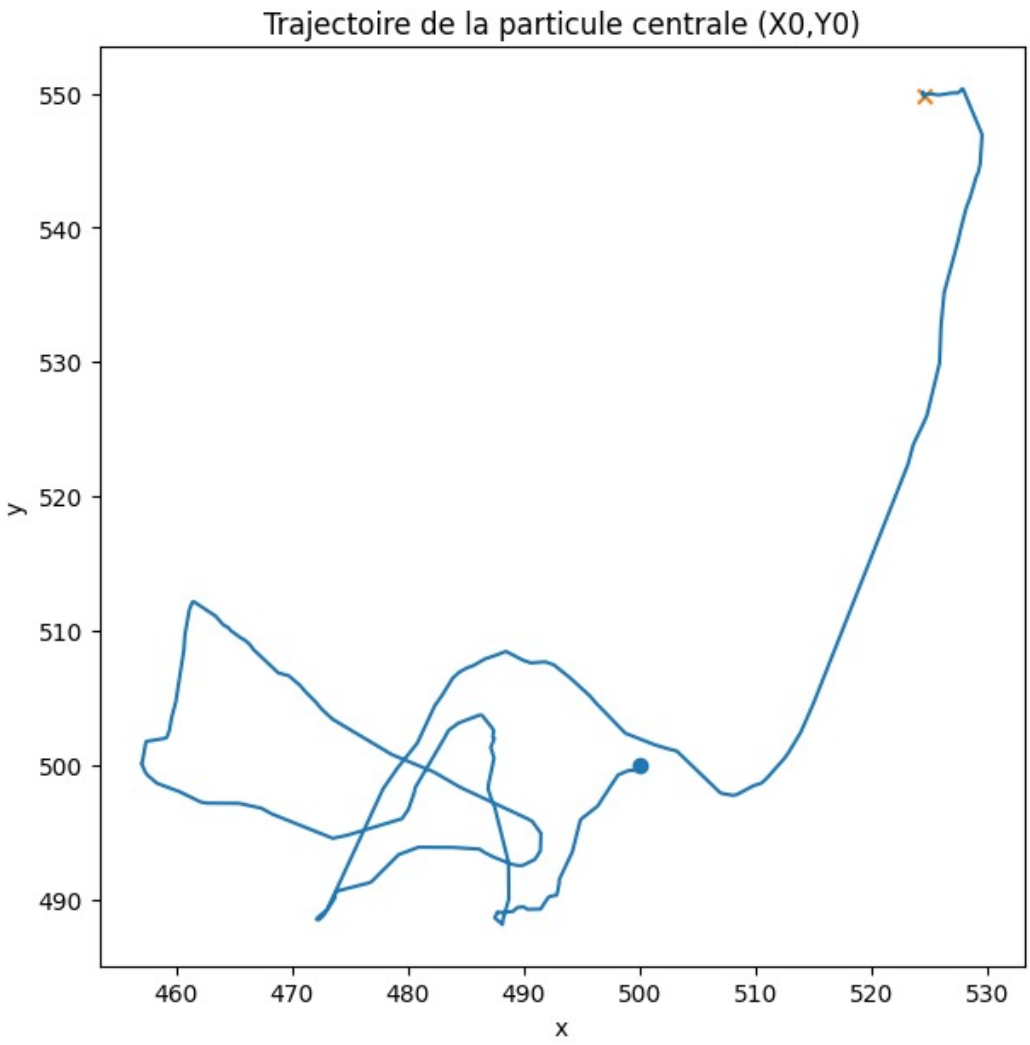


FIGURE 6 – Trajectoire d'une particule plus massive au sein d'un milieu composé de 500 particules

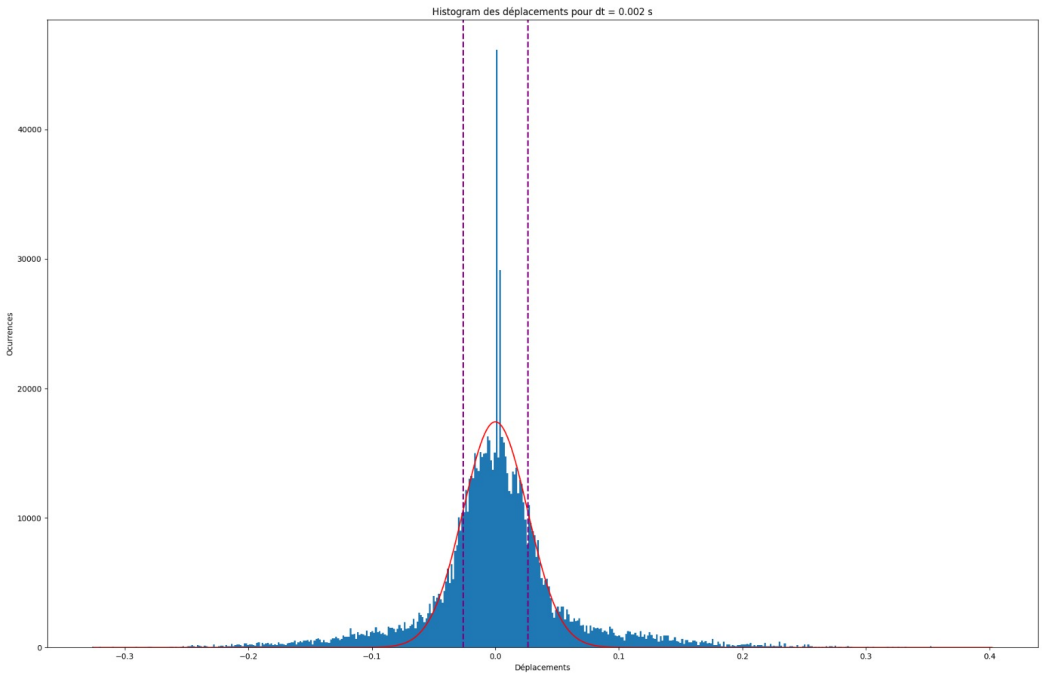


FIGURE 7 – Distribution des incréments pour un pas de temps fixé à 0,002 s

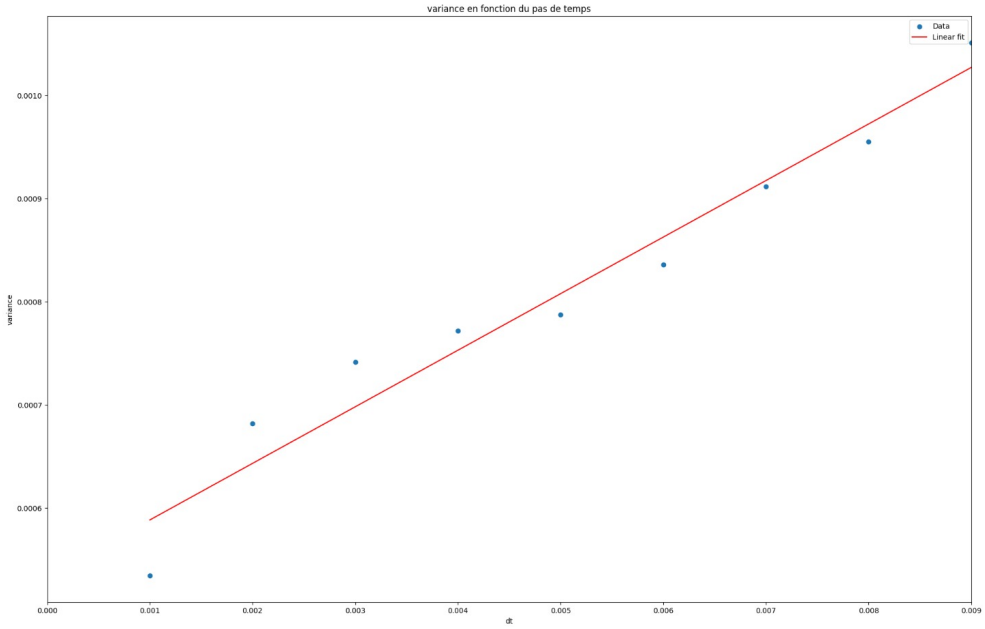


FIGURE 8 – Relation entre la variance et le pas de temps

8.2 Code

8.2.1 Structure principale du code

```python

import numpy as np

""" ici on definit ce qu'est une particule """

class Particule:

    def __init__(self):
        self.r = np.array([0.5, 0.5])
        self.v = np.asarray([0.9, 0.9], dtype=float)
        self.m = pow(10, -27)
        self.R = 0.0
        self.colls = 0


    def update(self, dt):
        self.r += self.v * dt
```

```python
import numpy as np
import random as rd
from part import Particule

"""

dans c_i.py on définit les différents condition initiales. On distingue
    broooown dans le cas du mouvement brownien ou une particule doite
    etre plus grosse, ou encore equilibre homogene, dans le cas général
    ou l'on veut démarer la simulation à l'équilibre

"""

def broooown(N, rayons_boite_x, rayons_boite_y, Temp, R, m, R_grosse =
    15, m_grosse = 50):

    x_max, y_max = rayons_boite_x*R_grosse, rayons_boite_y*R_grosse
    k_B = 100              # Constante de Boltzman

    # creation des partiucles
    print("Creation des particules")

    parts = []
    positions = np.zeros((N,2))
    rayons = np.zeros(N)

    sigma = np.sqrt(k_B * Temp / m)
    v_x = np.random.normal(0, sigma, N)
    v_y = np.random.normal(0, sigma, N)

    for i in range(N):

        p = Particule()

        # La grosse en premier
        if i == 0:
```

```
            p.R = R_grosse
            p.m = m if m_grosse is None else m_grosse

            a = np.array([x_max / 2.0, y_max / 2.0], dtype=float)

            p.r = a
            p.v = np.array([0.0, 0.0], dtype=float)

            positions[0] = a
            rayons[0] = p.R

        #Le reste
        else:
            p.R = R
            p.m = m

            a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)], dtype=float)

            dists = [np.linalg.norm(a - positions[j]) - p.R - rayons[j] for j in range(i)]

            m_min = min(dists)

            while m_min <= 0:
                a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)], dtype=float)
                dists = [np.linalg.norm(a - positions[j]) - p.R - rayons[j] for j in range(i)]

                m_min = min(dists)

            positions[i] = a
            rayons[i] = p.R

            p.r = a
            p.v = np.array([v_x[i], v_y[i]], dtype=float)

        parts.append(p)


    return parts, x_max, y_max



def equilibre_homog(N, R, rayons_boite_x, rayons_boite_y, m, Temp):

    x_max, y_max = rayons_boite_x*R, rayons_boite_y*R
    k_B = 1                # Constante de Boltzman

    # creation des partiucles
    print("creation des particules")

    parts = []
    positions = np.zeros((N,2))

    sigma = np.sqrt(k_B * Temp / m)
    v_x = np.random.normal(0, sigma, N)
```

```
    v_y = np.random.normal(0, sigma, N)

    for i in range(N):

        p = Particule()

        p.R = R

        if i==0:
            a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)])
            positions[0] = a
            p.r = a
        else:
            a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)])
            m = [np.linalg.norm(a - q) for q in positions]
            m_min = min(m)

            while m_min <= 2*p.R:
                a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)])
                m = [np.linalg.norm(a - q) for q in positions]
                m_min = min(m)

            positions[i] = a
            p.r = a

        p.v = np.array([v_x[i], v_y[i]])

        parts.append(p)

    # on calcul le pas de temps par rapport au rayon et la vitesse moyenne
    n_vitesses = [np.linalg.norm(p.v) for p in parts]
    n_v_mean = np.mean(n_vitesses)
    n_v_sigma = np.std(n_vitesses)
    v_repr = n_v_mean + n_v_sigma
    dt = R/(2*v_repr)

    return parts, dt, x_max, y_max
```

8.2.2 Animation

```

import anim
import heapq
from math import sqrt
import itertools
import numpy as np
import pandas as pd
from part import Particule
import sys
import c_i


"""
```

```python

main_eff.py est la partie du code qui calcul la simulation de maniere efficace.
Elle basée sur le principe où l'on prédit la prochain collision qui aura lieu dans le système et où l'on avance la simulation collision par collision.
Pour cela une liste de 'Event' est créé qui garde toutes le collision potentiel et qui donne celle qui arrive en première.

"""




class Event:
    def __init__(self, temps, indices, coll):
        self.items = list(indices)
        self.t = temps.copy()
        self.coll_count = coll.copy()

    def show(self):
        print(f"{self.items}, {self.t}")




counter = itertools.count()
events = []

def push_event(event: Event):
    heapq.heappush(events, (event.t, next(counter), event))

def pop_event():
    _, _, event = heapq.heappop(events)
    return event



# cette fonction est appelé lorsqu'une collision particule/paroi a lieu
def coll_paroi_eff(i,t, coll_count, parts, N, y_max, x_max):
    # extraire pos et vitesse particule
    vy = parts[i].v[1]
    vx = parts[i].v[0]

    ry = parts[i].r[1]
    rx = parts[i].r[0]

    ev = None
    # collision paroi
    if vy < 0:
        t_col = (parts[i].R - ry)/vy
        ev1 = Event(t_col+t, [i, -3], [parts[i].colls, 0])
        ev = ev1
        push_event(ev)
    elif vy > 0:
        t_col = (y_max - parts[i].R - ry)/vy
        ev1 = Event(t_col+t, [i, -4], [parts[i].colls, 0])
        ev = ev1
```

```
        push_event(ev)

    if vx < 0:
        t_col = (parts[i].R - rx)/vx
        ev = Event(t_col+t, [i, -1], [parts[i].colls, 0])
        push_event(ev)
    elif vx > 0:
        t_col = (x_max - parts[i].R - rx)/vx
        ev = Event(t_col+t, [i, -2], [parts[i].colls, 0])
        push_event(ev)

    # collision partiucles
    for j  in range(N):
        if j==i:
            continue
        dr = parts[j].r - parts[i].r
        dv = parts[j].v - parts[i].v

        c = dr[0]**2 + dr[1]**2 - (parts[j].R + parts[i].R)**2
        b = 2 * dr @ dv
        a = dv[0]**2 + dv[1]**2

        D = b**2 - 4*a*c

        if a > 0 and b <= 0 and D >= 0:             # collision uniquement possible si b strictement inf 0
            t_col = (-b-sqrt(D))/(2*a)
            if t_col < 0:
                continue
            ev = Event(t_col+t, [i, j], [parts[i].colls, parts[j].colls])
            push_event(ev)



# cette fonction est appelé lorsqu'une collision particule/particule a lieu
def coll_part_eff(ind1, ind2,t, coll_count, parts, N, y_max, x_max):
    for i in [ind1, ind2]:
        coll_paroi_eff(i,t, coll_count, parts, N, y_max, x_max)


def init_events(parts, N, y_max, x_max):
    t=0
    for i in range(N):
        # extraire pos et vitesse particule
        vy = parts[i].v[1]
        vx = parts[i].v[0]

        ry = parts[i].r[1]
        rx = parts[i].r[0]

        ev = None
        # collision paroi
        if vy < 0:
            t_col = (parts[i].R - ry)/vy
            ev1 = Event(t_col+t, [i, -3], [0,0])
            ev = ev1
```

```
            push_event(ev)
        elif vy > 0:
            t_col = (y_max - parts[i].R - ry)/vy
            ev1 = Event(t_col+t, [i, -4], [0,0])
            ev = ev1
            push_event(ev)

        if vx < 0:
            t_col = (parts[i].R - rx)/vx
            ev = Event(t_col+t, [i, -1], [0,0])
            push_event(ev)
        elif vx > 0:
            t_col = (x_max - parts[i].R - rx)/vx
            ev = Event(t_col+t, [i, -2], [0,0])
            push_event(ev)


        for j in range(i+1, N):
            # collision partiucles
            dr = parts[j].r - parts[i].r
            dv = parts[j].v - parts[i].v

            c = dr[0]**2 + dr[1]**2 - (parts[j].R + parts[i].R)**2
            b = 2 * dr @ dv
            a = dv[0]**2 + dv[1]**2

            D = b**2 - 4*a*c

            if a > 0 and b <= 0 and D >= 0:
                t_col = (-b-sqrt(D))/(2*a)
                if t_col < 0:
                    continue
                ev = Event(t_col+t, [i, j], [0,0])
                push_event(ev)



# fonction pour simulation et animation
def calc_simulation(x_max, y_max, T, N, parts):

    # variables pour stocker les données
    data_pos = []

    init_events(parts, N, y_max, x_max)


    # initialiser liste des evenements
    t = 0
    while t <= T:

        ev = pop_event()

        # on verifie d'abord si le event n'est pas obselete
        if ev.coll_count[0] == parts[ev.items[0]].colls and (ev.items[1]
            < 0 or ev.coll_count[1] == parts[ev.items[1]].colls):
            dt = ev.t - t

            # affichage de la progression du calcul
```

```
            t += dt
            print(t)

            row = {}
            row[f"time"] = t
            for i in range(N):
                parts[i].r += parts[i].v * dt
                row[f"X{i}"] = parts[i].r[0]
                row[f"Y{i}"] = parts[i].r[1]
            data_pos.append(row)


            # changements des vitesses des particules en collision
            ind1 = min(ev.items)
            ind2 = max(ev.items)
            if ind1 < 0:       # collision paroie
                if ind1 >= -2:     #gauche ou droite
                    parts[ind2].v[0] *= -1
                    parts[ind2].colls +=1
                else:                    # haut et bas
                    parts[ind2].v[1] *= -1
                    parts[ind2].colls +=1

                # predire prochaines collisions et ajouter a evenement
                coll_paroi_eff(ind2, t, ev.coll_count, parts, N, y_max,
                    x_max)
            else:                    # dans ce cas la collision est entre
                particules
                p1 = parts[ind1]
                p2 = parts[ind2]

                dr = p2.r - p1.r
                norme_dr = np.linalg.norm(dr)

                vcm = (p1.m * p1.v + p2.m * p2.v)/(p1.m + p2.m)

                u1 = p1.v - vcm
                u2 = p2.v - vcm

                n = dr/norme_dr

                u1n = u1 @ n
                u2n = u2 @ n

                # on change les vitesses
                p1.v -= 2.0*u1n*n
                p2.v -= 2.0*u2n*n

                p1.colls += 1
                p2.colls += 1

                # predire prochaines collisions
                coll_part_eff(ind1, ind2, t, ev.coll_count, parts, N,
                    y_max, x_max)


    print("préparation des animations")

```

```
    R = [p.R for p in parts]
    anim.anim_direct(x_max, y_max, R, dt, T, N, data_pos)
    return data_pos




# calcul de la simulation pour le mouvement brownien
def calc_simulation_brown(x_max, y_max, T, N, parts):

    # variables pour stocker les données
    data_pos = []

    init_events(parts, N, y_max, x_max)


    # initialiser liste des evenements
    t = 0
    t_prev_col_big = 0
    while t <= T:

        ev = pop_event()

        # on verifie d'abord si l'évenement est obselete
        if ev.coll_count[0] == parts[ev.items[0]].colls and (ev.items[1]
            < 0 or ev.coll_count[1] == parts[ev.items[1]].colls):
            dt = ev.t - t

            t += dt

            for i in range(N):
                parts[i].r += parts[i].v * dt

            # on enregesitre que quand grosse particule et que quand il
               y a collision
            if 0 in ev.items:
                row = {}
                row[f"time"] = t
                t_prev_col_big = t
                row[f"X0"] = parts[0].r[0]
                row[f"Y0"] = parts[0].r[1]
                data_pos.append(row)


            # changer les vitesses des particules en collision
            ind1 = min(ev.items)
            ind2 = max(ev.items)
            if ind1 < 0:       # collision paroie
                if ind1 >= -2:     #gauche ou droite
                    parts[ind2].v[0] *= -1
                    parts[ind2].colls +=1
                else:                    # haut et bas
                    parts[ind2].v[1] *= -1
                    parts[ind2].colls +=1

                # predire prochaines collisions et ajouter a evenement
                coll_paroi_eff(ind2, t, ev.coll_count, parts, N, y_max,
                   x_max)
```

```
            else:                    # dans ce cas la collision est entre deux partiucles
                p1 = parts[ind1]
                p2 = parts[ind2]

                dr = p2.r - p1.r
                norme_dr = np.linalg.norm(dr)

                vcm = (p1.m * p1.v + p2.m * p2.v)/(p1.m + p2.m)

                u1 = p1.v - vcm
                u2 = p2.v - vcm

                n = dr/norme_dr

                u1n = u1 @ n
                u2n = u2 @ n

                # on change les vitesses
                p1.v -= 2.0*u1n*n
                p2.v -= 2.0*u2n*n

                p1.colls += 1
                p2.colls += 1

                # predire prochaines collisions
                coll_part_eff(ind1, ind2, t, ev.coll_count, parts, N, y_max, x_max)

    return data_pos
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import time

"""

anim.py nous permet d'animer la simulation.

"""

def anim_direct(x_max, y_max, R, dt, T, N_params, data_pos):
    global ani

    data = pd.DataFrame(data_pos)
    N_data = data.shape[1]//2

    if N_params != N_data:
        print(f"ERREUR:: N correspond pas au nombre de particules")
        N = min(N_params, N_data)
    else :
        N = N_params

    n_frames = data.shape[0]
    print(n_frames)
```

```
    times = np.arange(n_frames) * dt

    # Figure
    fig, ax = plt.subplots(figsize=(13,13))
    ax.set_aspect("equal", adjustable="box")
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_title("Particules de gaz dures en 2D")

    ax.plot([0, x_max, x_max, 0, 0], [0, 0, y_max, y_max,0], lw=2)

    # Particules
    circles = []

    spec_col = 0
    for i in range(N):
        if i == spec_col :
            color = "red"
        else:
            color="blue"
        circ = plt.Circle((data.loc[0, f"X{i}"], data.loc[0, f"Y{i}"]),
           R[i], color=color, fill=True,)
        ax.add_patch(circ)
        circles.append(circ)


    def t_init():
        for i, circ in enumerate(circles):
            circ.center = (data.loc[0, f"X{i}"], data.loc[0, f"Y{i}"])
        return circles

    def t_update(frame):
        for i, circ in enumerate(circles):
            x, y = data.loc[frame, f"X{i}"], data.loc[frame, f"Y{i}"]
            circ.center = (x, y)
        return circles

    epsilon = dt*1000
    ani = FuncAnimation(fig, t_update, frames=n_frames, init_func=t_init
       , interval=epsilon, blit=True)
    plt.show()
```

8.2.3 Distribution de Maxwell-Boltzmann

```
""" SIMULATEUR BOLTZMAN """


#Importation des modules
from itertools import combinations
import numpy as np
import pandas as pd
import random as rd
import matplotlib.pyplot as plt


#Import Particules
class Particule:
```

```python

    def __init__(self):
        self.r = np.array([0.5, 0.5])
        self.v = np.array([0.9, 0.9])
        self.m = pow(10, -27)
        self.R = 0.05


    def update(self, dt):
        self.r += self.v * dt


#fonction simulation
def simulation_boltzman(N=4, Temp=300, T=10, v_set = 1000):


    #Paramètres de la simulation
    fact_dim = pow(10,9)
    T = T #temps de la simulation en seconde
    N = N #Nombre de particules
    """ marche que pour un gaz homogène """
    R = pow(10.0, -9) * fact_dim #Rayon des particules
    x_max, y_max = 50*R, 50*R #dimension de la boite
    m = pow(10.0, -27) #Masse des particules
    Temp = Temp #Température en Kelvin (sert juste d'échelle initiale)
    k_B = 1.38e-23 #Constante de Boltzmann


    #Création des particules
    print("Création des particules...")
    parts = []
    positions = []

    positions = np.zeros((N,2))


    for i in range(N):
        p = Particule()
        p.R = R


        if i ==0:
            a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)])
            positions[0] = a
            p.r = a
        else:
            a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)])
            m_test = [np.linalg.norm(a-q) for q in positions]
            m_min = min(m_test)


            while m_min <= 2*p.R:
                a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)])
                m_test = [np.linalg.norm(a-q) for q in positions]
                m_min = min(m_test)
```

```python


            positions[i] = a
            p.r = a

        v_x = rd.uniform(-v_set, v_set)
        v_y = rd.uniform(-v_set, v_set)
        p.v = np.array([v_x, v_y])

        parts.append(p)


    #Calcul du pas de temps
    n_vitesses = [np.linalg.norm(p.v) for p in parts]
    n_v_mean = np.mean(n_vitesses)
    v_repr = n_v_mean
    dt = R / (2 * v_repr)
    print(f"Pas de temps calculé : {dt} s")

    # Température mesurée
    def temperature_instantanee():
        v2 = np.mean([np.dot(p.v, p.v) for p in parts])
        return m * v2 / (2*k_B)


    # Distribution Maxwell 2D
    def maxwell_2D(v, T):
        return (m/(k_B*T)) * v * np.exp(-m*v**2/(2*k_B*T))


    # Fonction de collision sur parois
    def coll_paroi(p):

        r_new = p.r + p.v * dt


        if r_new[0] < p.R:
            p.v[0] *= -1
            r_new[0] = p.R

        elif r_new[0] > x_max - p.R:
            p.v[0] *= -1
            r_new[0] = x_max - p.R


        if r_new[1] < p.R:
            p.v[1] *= -1
            r_new[1] = p.R

        elif r_new[1] > y_max - p.R:
            p.v[1] *= -1
            r_new[1] = y_max - p.R


        p.r = r_new


    #Fonction de collision entre particules
```

```
def coll_part(p1, p2):
    dr = p2.r - p1.r
    norme_dr = np.linalg.norm(dr)


    if norme_dr <= p1.R + p2.R:
        vcm = (p1.m * p1.v + p2.m * p2.v) / (p1.m + p2.m)
        u1 = p1.v - vcm
        u2 = p2.v - vcm
        n = dr / norme_dr
        u1n = u1 @ n
        u2n = u2 @ n
        p1.v -= 2*u1n*n
        p2.v -= 2*u2n*n

    overlap = (p1.R + p2.R) - norme_dr
    if overlap > 0:
        w1 = p2.m / (p1.m + p2.m)
        w2 = p1.m / (p1.m + p2.m)
        p1.r -= w1 * overlap * n
        p2.r += w2 * overlap * n


# Execution de la simulation
print("Lancement de la simulation...")
t = 0
step = 0
record_interval = 2000
N_simu = int(T / record_interval)
n_simu = 1


while t <= T:


    for i in range(N):
        coll_paroi(parts[i])

    for a, b in combinations(parts, 2):
        coll_part(a, b)

    n_suivies = [np.linalg.norm(p.v) for p in parts]


    if step % record_interval == 0:


        T_inst = temperature_instantanee()

        v_axis = np.linspace(0, max(n_suivies)*1.2, 400)
        f_MB = maxwell_2D(v_axis, T_inst)


        print(f"Screen n  {step // record_interval + 1} | t = {t:.2e}s | T = {T_inst:.2f} K")


        plt.figure(figsize=(8, 6))
```

```
            plt.hist(n_suivies, bins=50, density=True, alpha=0.7, color="pink", label="Simulation")
            plt.plot(v_axis, f_MB, color="purple", linewidth=2, label="Maxwell-Boltzmann (2D)")
            plt.title(f"Distribution des vitesses (étape {n_simu})")
            plt.xlabel("Vitesse (m/s)")
            plt.ylabel("Densité de probabilité")
            plt.legend()
            plt.grid()
            plt.show()


            n_simu += 1

        t += dt
        step += 1

    print("Simulation terminée")


simulation_boltzman(200, 300, 20, 1000)
```

8.2.4 Pression Numérique

```

""" SIMULTATION PRESSION REPETE """
"""

Ce code est une version modifié du code 'main' précédent. Il permet de centraliser en une fonction le fait de lancer une simulation, ce qui nous sera très utile pour les 2 codes qui suivent, afin d'obtenir des données sur un grand échantillon.

"""

#Importation des modules
from itertools import combinations
import numpy as np
import pandas as pd
import random as rd
import matplotlib.pyplot as plt


#Import Particules
class Particule:

    def __init__(self):
        self.r = np.array([0.5, 0.5])
        self.v = np.array([0.9, 0.9])
        self.m = pow(10, -27)
        self.R = 0.05




    def update(self, dt):
        self.r += self.v * dt
```

```python


#fonction simulation
def simulation_pression_rep(N=4, Temp=300, T=10):
    """ Créer les données nécessaires au calcul de pression """


    #Paramètres de la simulation
    fact_dim = pow(10,9)
    T = T #temps de la simulation en seconde
    N = N #Nombre de particules
    """ marche que pour un gaz homogène """
    R = pow(10.0, -9) * fact_dim #Rayon des particules
    x_max, y_max = 50*R, 50*R #dimension de la boite
    m = pow(10.0, -27) #Masse des particules
    Temp = Temp #Température en Kelvin
    k_B = 1.38e-23 #Constante de Boltzmann


    #Création des particules
    parts = []
    positions = []


    sigma = np.sqrt(k_B * Temp / m)
    v_x = np.random.normal(0, sigma, N)
    v_y = np.random.normal(0, sigma, N)


    positions = np.zeros((N,2))


    for i in range(N):
        p = Particule()
        p.R = R


        if i ==0:
            a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)])
            positions[0] = a
            p.r = a
        else:
            a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)])
            m = [np.linalg.norm(a-q) for q in positions]
            m_min = min(m)


            while m_min <= 2*p.R:
                a = np.array([rd.uniform(p.R, x_max - p.R), rd.uniform(p.R, y_max - p.R)])
                m = [np.linalg.norm(a-q) for q in positions]
                m_min = min(m)


            positions[i] = a
            p.r = a
```

```python


        p.v = np.array([v_x[i], v_y[i]])

        parts.append(p)


    #Calcul du pas de temps
    n_vitesses = [np.linalg.norm(p.v) for p in parts]
    n_v_mean = np.mean(n_vitesses)
    v_repr = n_v_mean + sigma
    dt = R / (2 * v_repr)


    #Data pour la pression
    v_collision = []
    m_collision = []



    def coll_paroi_pression(p):

        r_new = p.r + p.v * dt


        # Collision avec une paroi verticale
        if r_new[0] < p.R: #Collision avec la paroi de gauche
            v_collision.append(abs(p.v[0]))
            m_collision.append(p.m)
            p.v[0] *= -1
            r_new[0] = p.R

        elif r_new[0] > x_max - p.R: # Collision avec la paroi de droite
            v_collision.append(abs(p.v[0]))
            m_collision.append(p.m)
            p.v[0] *= -1
            r_new[0] = x_max - p.R


        #Collision avec une paroi horizontale
        if r_new[1] < p.R: # Collision avec la paroi du bas
            v_collision.append(abs(p.v[1]))
            m_collision.append(p.m)
            p.v[1] *= -1
            r_new[1] = p.R

        elif r_new[1] > y_max - p.R: # Collision avec la paroi du haut
            v_collision.append(abs(p.v[1]))
            m_collision.append(p.m)
            p.v[1] *= -1
            r_new[1] = y_max - p.R


        p.r = r_new



    def coll_part(p1, p2):
```

```
        dr = p2.r - p1.r
        norme_dr = np.linalg.norm(dr)


        if norme_dr <= p1.R + p2.R: #Collistion de particules
            vcm = (p1.m * p1.v + p2.m * p2.v) / (p1.m + p2.m)
            u1 = p1.v - vcm
            u2 = p2.v - vcm
            n = dr / norme_dr
            u1n = u1 @ n
            u2n = u2 @ n
            p1.v -= 2*u1n*n
            p2.v -= 2*u2n*n

        overlap = (p1.R + p2.R) - norme_dr
        if overlap > 0:
            w1 = p2.m / (p1.m + p2.m)
            w2 = p1.m / (p1.m + p2.m)
            p1.r -= w1 * overlap * n
            p2.r += w2 * overlap * n


    # Execution de la simulation
    t = 0
    while t <= T:
        # Collisions avec les parois plus enregistrement de la pression
        for i in range(N):
            coll_paroi_pression(parts[i])

        # Collisions entre les particules
        for a, b in combinations(parts, 2):
            coll_part(a, b)


        t += dt


    #Calcul de la Pression Numérique


    #Préparation des données
    v_collisions = np.array(v_collision)
    m_collisions = np.array(m_collision)


    #Compteur de pression
    total_q = 0 #quantité de mouvement total subis par les parois
    nb = len(v_collisions)
    q = []
    for i in range(nb):
        q_i = 2 * m_collisions[i] * v_collisions[i]
        q.append(q_i)
    total_q = np.sum(q)

    #Calcul de la pression résultante en 2D
    L = 2 * (x_max + y_max)
    P = total_q / (L * T)

```

```python

    # Affichage des résultats
    return P
```

```python
"""
Ce code a permis de tracer l'histogramme de la distribution des pressions calculées numériquement
"""

def hist_pression(N=4, Temp=300, T = 4, Rep = 4):


    #Donnée simulation
    fact_dim = pow(10,9)
    k_B = 1.38e-23 #Constante de Boltzman
    R = pow(10.0, -9) * fact_dim #Rayon des particules
    x_max, y_max = 50*R, 50*R #dimension de la boite

    # Calcul analytique de la pression
    P_analytique = (N * k_B * Temp) / (x_max * y_max - N * np.pi * R**2)
    print(f"Pression analytique : {P_analytique:.2e} Pa")


    #Liste des pressions calculées
    list_pression = []
    for k in range(Rep):
        P = simulation_pression_rep(N, Temp, T)
        list_pression.append(P)
        print(f"Répétition {k+1}/{Rep} : Pression simulée = {P:.2e} Pa")
    data = np.array(list_pression)


    #Création de l'histogramme
    data = np.array(list_pression)


    bins = int(np.sqrt(len(data)))


    plt.figure(figsize=(8, 5))
    plt.hist(data, bins=bins, density=True, alpha=0.6, edgecolor="black")
    plt.axvline(P_analytique, color="red", linestyle="--", linewidth=2, label="Pression analytique")
    plt.xlabel("Pression")
    plt.ylabel("Densité de probabilité")
    plt.title("Distribution des pressions simulées")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()


hist_pression(30, 300, 20, 100)
```

```python
"""
Ce code permet de tracer le graphe de la pression calculé en fonction de la température

```

```python
"""

def temp_pression(N, min_temp, max_temp, T):


    #Donnée du problème
    fact_dim = pow(10,9)
    R = pow(10.0, -9) * fact_dim #Rayon des particules
    x_max, y_max = 50*R, 50*R #dimension de la boite
    k_B = 1.38e-23 #Constante de Boltzmann
    Temp = np.arange(min_temp, max_temp, 1)


    #Pression Analytique en fonction de la température
    P_ana = (N * k_B * Temp) / (x_max * y_max - N * np.pi * pow(R, 2))


    #Calcul de la pression numérique en fonction du temps
    list_pression = []
    for k in Temp:
        P = simulation_pression_rep(N, k, T)
        list_pression.append(P)
        print(f"Température : {k} K, Pression : {P} Pa")
    P_num = np.array(list_pression)


    #Graphique
    plt.figure(figsize=(10,6))
    plt.plot(Temp, P_num, label = "Pression numérique", color = "purple")
    plt.plot(Temp, P_ana, label = "Pression analytique", color = "orange")
    plt.title("Pression en fonction de la température")
    plt.xlabel('Température (en K)')
    plt.ylabel('Pression (en Pa)')
    plt.legend()
    plt.grid()
    plt.show()


temp_pression(50, 1, 300, 20)  # Vous pouvez retourner vaquer à vos occupations, il ne vous reste que 25h à attendre...
```

8.2.5 Mouvement Brownien

```python
import numpy as np
import c_i
from main_eff import calc_simulation_brown

"""

dans brown_reps.py on répete un certain nombre de foix la simulation d'un mouvement brownien en gardant uniquement les positionts et leurs temps associés

"""

```

```python
N = 500          # nombre de particules
T = 30           # temps

R = 1            # rayon petite partiucle
m = 1            # masse petite particule
Temp = 300                    # Température en Kelvin

R_grosse = 100   # rayon grosse particule

# variables et fichiers pour stocker les données
data_pos = {}
lengths = []
times = {}

rep = 10         # nombre de répetitions



# execution de la simulation
for i in range(rep):
    print(f"C'est la simulation {i+1}/{rep}")
    parts, x_max, y_max = c_i.broooown(N, 10, 10, Temp, R, m, R_grosse)

    # execution simulation avec extraction données grosse part
    data = calc_simulation_brown(x_max, y_max, T, N, parts)

    data_pos[f"X{i}"] = [row["X0"] for row in data]
    data_pos[f"Y{i}"] = [row["Y0"] for row in data]
    times[f"X{i}"] = [row["time"] for row in data]
    times[f"Y{i}"] = [row["time"] for row in data]
    lengths.append(len(data))    # number of rows


# pour toutes les colonnes de data_pos ai la meme taille
max_len = max(lengths)
for k, v in data_pos.items():
    if len(v) < max_len:
        data_pos[k] = v + [np.nan] * (max_len - len(v))

# pour que toutes les colonnes de times ai la meme taille
max_len = max(lengths)
for k, v in times.items():
    if len(v) < max_len:
        times[k] = v + [np.nan] * (max_len - len(v))


pos_new = pd.DataFrame(data_pos)
time_new = pd.DataFrame(times)
# sauvegarde de données sous format excel
pos_new.to_excel("mvt_brown_pos.xlsx", index=False)
time_new.to_excel("mvt_brown_time.xlsx", index=False)
```

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import curve_fit
import math
```

```python


"""

dans brown_coef.py on vérifie le caractère brownien de la trajectoire de la grosse particule en calculant les distribution des deplacements pour differents
pas de temps. Et en affichant la variance d'une distribution en fonction du pas de temps choisi

"""


# fonction modèle
def gauss(x, A, mu, sigma):
    return A / sigma * np.exp(-(x-mu)**2 / (2*sigma**2))

#Importation du fichier
filename_pos = "mvt_brown_pos.xlsx"
filename_time = "mvt_brown_time.xlsx"
df = pd.read_excel(filename_pos)
time = pd.read_excel(filename_time)
df = df.astype("float64")


delta = []

vars = []
d_temps = []


# ici on calcul étant donné un pas de temps les positions de la particule pour chaque incrémentation du pas de temps
def resample_positions_with_time_df(pos_df: pd.DataFrame, time_df: pd.DataFrame, dt_uniform: float):
    # Build per-column time arrays + find common end time
    t_end = np.inf
    t_col_map = {}

    for col in pos_df.columns:
        t_col = time_df[col].to_numpy(dtype=float)
        x_col = pos_df[col].to_numpy(dtype=float)

        # on enlève les cellules vides
        mask = np.isfinite(t_col) & np.isfinite(x_col)
        t_col = t_col[mask]
        x_col = x_col[mask]

        if t_col.size < 2:
            raise ValueError(f"{col} trop petite")

        # l'orgine des temps est fixé à 0 s
        t_col = t_col - t_col[0]

        t_col_map[col] = (t_col, x_col)
        t_end = min(t_end, t_col[-1])    # on sauvegarde le temps a la fin pour chaque colone et on garde le plus petit

```

```
    # création de la nouvelle liste de temps avec le pas de temps constant
    # on s'arrète a t_end pour toutes les colonnes
    t_new = np.arange(0.0, t_end + 1e-12, dt_uniform)


    out = {"t": t_new}

    # calcule des position pour chaque pas de temps par interpolation
    for col in pos_df.columns:
        t_col, x_col = t_col_map[col]
        out[col] = np.interp(t_new, t_col, x_col)

    return pd.DataFrame(out)



for d in range(1,10):
    dt_uniform = d*0.001 # on fixe notre pas de temps
    data = resample_positions_with_time_df(df, time, dt_uniform)
    data.iloc[:, 1:]     # on s'interesse uniquement au positions

    # on calcul les incrémentation pour chaque colonne avec le saut de temps approprié
    for col_name in data.columns:
        for i in range(0, data[col_name].size - d, 1):
            temp = data.loc[i+d, col_name]
            temp1 = data.loc[i, col_name]
            delta.append(temp - temp1)

    # creation de l'histogramme
    data_hist = np.array(delta)
    data_hist = data_hist[np.isfinite(data_hist)]
    n_bins = 500
    counts, bin_edges = np.histogram(data_hist, bins=n_bins)

    # curve fit à la gaussienne
    bin_centers = 0.5*(bin_edges[1:] + bin_edges[:-1])  # on prend le centre des bins pour les valeurs de x
    p_opt, p_cov = curve_fit(
        gauss,
        bin_centers,
        counts,
        p0=[counts.max(), data_hist.mean(), data_hist.std()]
    )

    # extraction des paramètres de la gaussienne
    mu = p_opt[1]
    sigma = p_opt[2]

    var = pow(sigma, 2)
    # on garde à chaque foix la variance et le pas de temps utilisé
    vars.append(var)
    d_temps.append(dt_uniform)


    # on affiche pour un pas de temps au choix la gaussienne
    if d==2:
```

```
        x_fit = np.linspace(bin_centers.min(), bin_centers.max(), 1000)
        plt.hist(data_hist, bins=n_bins)
        plt.plot(x_fit, gauss(x_fit, *p_opt), 'r')
        plt.axvline(x = mu + sigma, color='purple', linestyle='--',
            linewidth=2)
        plt.axvline(x = mu -sigma, color='purple', linestyle='--',
            linewidth=2)

        plt.xlabel('Déplacements')
        plt.ylabel('Ocurrences')
        plt.title('Histogram des déplacements pour dt = 0.002 s')
        plt.show()


# regression linéaire variance en fonction du pas de temps
coef_dir, ord_origine = np.polyfit(d_temps, vars, 1)

plt.scatter(d_temps, vars, label="Data")

x_line = np.linspace(min(d_temps), max(d_temps), 100)
y_line = coef_dir * x_line + ord_origine
plt.xlabel('dt')
plt.ylabel('variance')
plt.title('variance en fonction du pas de temps')
plt.xlim(0, max(d_temps))
plt.plot(x_line, y_line, color="red", label="Linear fit")
plt.legend()
plt.show()

print("slope =", coef_dir)
print("intercept =", ord_origine)
```

```
from main_eff import calc_simulation_brown
import c_i

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


"""

dans brown_anal.py on calcul et on affiche la trajectoire totale d'une
    particule qui suit un mouvement brownien

"""


N = 500
T = 10

R = 1
m = 1
Temp = 300                    # Température en Kelvin

R_grosse = 100

# initialisation du système
```

```
parts, x_max, y_max = c_i.broooown(N, 10, 10, Temp, R, m, R_grosse)


# calcul de la simulaiton
data = calc_simulation_brown(x_max, y_max, T, N, parts)
df = pd.DataFrame(data)

# trajectoire de la particule centrale
x = df["X0"].to_numpy()
y = df["Y0"].to_numpy()

# déplacement par rapport à la position initiale
dx = x - x[0]
dy = y - y[0]
r = np.sqrt(dx**2 + dy**2)

# Mouvement Brownien
plt.figure(figsize=(7,7))
plt.plot(x, y)
plt.scatter([x[0]], [y[0]], marker="o")       # départ
plt.scatter([x[-1]], [y[-1]], marker="x")     # arrivée
plt.xlabel("x")
plt.ylabel("y")
plt.title("Trajectoire de la particule centrale (X0,Y0)")
plt.show()
```